AI/ML FRAMEWORKS

Keras

Build, train, and ship neural networks fast with Keras 3's multi-backend API — from Sequential stacks to CNNs, RNNs, callbacks, and deployment.

keras · deep-learning · neural-networks · keras-3 · cnn · rnn · model-training

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. 1. What Keras Is and Why It Matters
2. 2. Setup and Getting Started
3. 3. The Three Ways to Build a Model
4. 4. Layers: The Building Blocks
5. 5. Compile, Fit, Evaluate, Predict
6. 6. Callbacks: Hooks Into Training
7. 7. Common Architectures
8. 8. Regularization: Fighting Overfitting
9. 9. Saving, Loading, and Deployment
10. 10. Worked Mini-Example: MNIST End-to-End
11. 11. Cheat Sheet
12. 12. Best Practices and Common Pitfalls
13. 13. Interview Questions & Answers
14. 14. Further Learning

Keras is the high-level deep-learning API that lets you go from an idea to a trained neural network in a handful of readable lines. With Keras 3 it became truly **multi-backend** — the same model code runs on TensorFlow, JAX, or PyTorch — so you write once and choose the engine that best fits your deployment, hardware, or team. It is the fastest way to be productive in deep learning while staying close enough to the metal for serious research.

1. What Keras Is and Why It Matters

Keras is a **declarative, layer-oriented API** for building and training neural networks. You compose *layers* into a *model*, attach an optimizer, loss, and metrics with `compile()`, then drive the training

loop with `fit()`. The library handles autodifferentiation, batching, device placement, and the gradient-descent loop so you focus on architecture and data.

Keras 3 (the current major line, GA since late 2023 and the default `keras` package on PyPI in 2024-2026) is a rewrite on top of a thin **backend abstraction**. The *same* `keras.layers` and `Model` code is executed by whichever backend you select — **TensorFlow**, **JAX**, or **PyTorch**. This decouples your model definition from the numerical engine.

READABLE

Models read like the diagrams on the whiteboard. Stacking layers is a list; branching is a function call.

PORTABLE

One backend switch (an env var) moves training between TF, JAX, and PyTorch with no model rewrite.

BATTERIES-INCLUDED

Optimizers, losses, metrics, callbacks, preprocessing layers, and saving are all first-class.

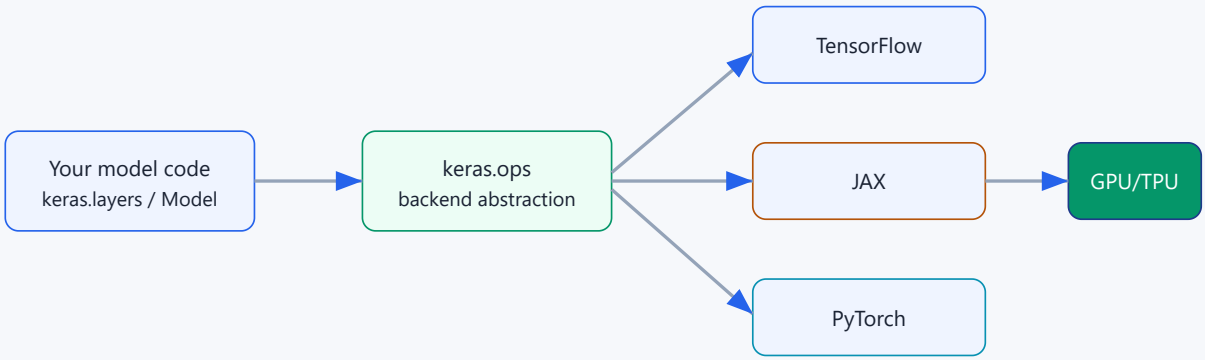


Fig 1 — Keras 3 architecture: one model definition, three interchangeable numerical backends.

> **Keras vs tf.keras**
> Historically Keras lived inside TensorFlow as `tf.keras`. Keras 3 is a standalone `pip install keras` package that sits *above* the backends. New code should `import keras` directly and select a backend, rather than relying on `tensorflow.keras`.

2. Setup and Getting Started

Install Keras 3 plus at least one backend. The backend is chosen at import time via the `KERAS_BACKEND` environment variable (default is `tensorflow`).

BASH

```bash
# Install Keras 3 and the backends you want
pip install --upgrade keras

# Pick ONE (or more) backend engine:
pip install tensorflow        # TensorFlow backend (default)
pip install "jax[cpu]"        # JAX backend (or jax[cuda12] for GPU)
pip install torch             # PyTorch backend

# Select the backend BEFORE importing keras:
export KERAS_BACKEND=jax      # or "tensorflow" / "torch"
python train.py
```

PYTHON

```python
# You can also set the backend in code, but it MUST come before `import keras`.
import os
os.environ["KERAS_BACKEND"] = "jax"

import keras
print(keras.__version__)        # e.g. 3.x
print(keras.backend.backend())  # "jax"
```

✦ **Reproducibility**

Call `keras.utils.set_random_seed(42)` once at the top of your script to seed Python, NumPy, and the active backend together.

3. The Three Ways to Build a Model

Keras offers three model-building styles, ordered from simplest to most flexible. Pick the lowest-ceremony option that your architecture allows.

SEQUENTIAL

A linear stack of layers. One input, one output, no branching. Great for MLPs and simple CNNs.

FUNCTIONAL

Layers called like functions on tensors. Supports multiple inputs/outputs, branches, and shared layers — the workhorse for most research.

SUBCLASSING

Write a `Model` subclass with an imperative `call()`. Maximum flexibility for dynamic control flow and custom training.

3.1 Sequential API

PYTHON

```python
import keras
from keras import layers

model = keras.Sequential([
    layers.Input(shape=(784,)),      # explicit input shape (28x28 flattened)
    layers.Dense(256, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(10, activation="softmax"),  # 10-class classifier
])

model.summary()  # prints layer table, output shapes, and parameter counts
```

3.2 Functional API

The functional API treats layers as callables on symbolic **KerasTensor** objects. You explicitly wire the dataflow graph, which makes branches, residual connections, and multi-input models natural.

PYTHON

```python
from keras import layers, Model

inputs = keras.Input(shape=(784,))
x = layers.Dense(256, activation="relu")(inputs)
x = layers.Dropout(0.3)(x)

# A residual-style branch: add a skip connection
skip = layers.Dense(64, activation="relu")(x)
x = layers.Dense(64, activation="relu")(skip)
x = layers.add([x, skip])                       # element-wise merge

outputs = layers.Dense(10, activation="softmax")(x)
model = Model(inputs=inputs, outputs=outputs, name="mlp_with_skip")
```

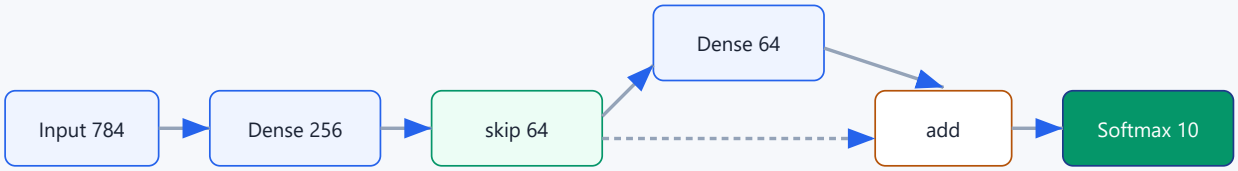


Fig 2 — Functional dataflow graph with a skip connection (dashed) merged by an `add` layer.

3.3 Model Subclassing

PYTHON

```python
class MLP(keras.Model):
    def __init__(self, units=256, classes=10):
        super().__init__()
        self.d1 = layers.Dense(units, activation="relu")
        self.drop = layers.Dropout(0.3)
        self.out = layers.Dense(classes, activation="softmax")

    def call(self, x, training=False):
        x = self.d1(x)
        x = self.drop(x, training=training)   # dropout only active in training
        return self.out(x)

model = MLP()
```

> ⚠ **Pass training through**
> In subclassed models you must thread the `training` flag into layers like Dropout and BatchNormalization yourself, or they will behave the same in train and inference. The Sequential and Functional APIs handle this automatically.

4. Layers: The Building Blocks

Every layer is a callable object with trainable *weights* and a `call()` that transforms tensors. Weights are created lazily the first time the layer sees an input shape (or eagerly if you pass `Input`).

Layer	Purpose	Key arguments
`Dense`	Fully-connected / linear	`units`, `activation`
`Conv2D`	2D convolution (images)	`filters`, `kernel_size`, `strides`, `padding`
`MaxPooling2D`	Spatial downsampling	`pool_size`
`LSTM` / `GRU`	Recurrent sequence modeling	`units`, `return_sequences`
`Embedding`	Integer tokens to dense vectors	`input_dim`, `output_dim`
`BatchNormalization`	Normalize activations	`axis`, `momentum`
`Dropout`	Stochastic regularization	`rate`
`MultiHeadAttention`	Transformer attention	`num_heads`, `key_dim`

4.1 Preprocessing layers

Keras ships preprocessing layers (`Normalization`, `Rescaling`, `TextVectorization`, image augmentation) that become *part of the model*. This means preprocessing travels with the saved model — no train/serve skew.

PYTHON

```python
# Adapt a Normalization layer to your data's mean/variance, then bake it in.
norm = layers.Normalization()
norm.adapt(x_train)              # computes mean & variance from data

inputs = keras.Input(shape=(10,))
x = norm(inputs)                 # normalization is now inside the model
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)
```

5. Compile, Fit, Evaluate, Predict

The training lifecycle is four verbs. `compile()` configures *how* to learn, `fit()` runs the loop, `evaluate()` scores held-out data, and `predict()` produces outputs.



PYTHON

```python
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",   # integer labels, multiclass
    metrics=["accuracy"],
)

history = model.fit(
    x_train, y_train,
    validation_split=0.1,     # hold out 10% for validation each epoch
    epochs=20,
    batch_size=128,
    verbose=2,
)

test_loss, test_acc = model.evaluate(x_test, y_test)
probs = model.predict(x_test[:5])    # shape (5, 10) class probabilities
```



Fig 3 — One training step inside `fit()`: forward pass, loss, backprop, weight update.

5.1 Choosing a loss

Task	Output layer	Loss
Binary classification	`Dense(1, "sigmoid")`	`binary_crossentropy`
Multiclass (integer labels)	`Dense(k, "softmax")`	`sparse_categorical_crossentropy`
Multiclass (one-hot labels)	`Dense(k, "softmax")`	`categorical_crossentropy`
Regression	`Dense(1)` (linear)	`mse` / `mae` / `huber`

6. Callbacks: Hooks Into Training

Callbacks are objects passed to `fit()` that fire at defined points (epoch start/end, batch end). They power early stopping, checkpointing, learning-rate schedules, and logging without touching your model code.

PYTHON

```
callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_loss", patience=5, restore_best_weights=True),
    keras.callbacks.ModelCheckpoint(
        "best.keras", monitor="val_loss", save_best_only=True),
    keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss", factor=0.5, patience=3),
    keras.callbacks.TensorBoard(log_dir="./logs"),
]

model.fit(x_train, y_train, validation_split=0.1,
          epochs=100, callbacks=callbacks)
```

✦ **EarlyStopping + restore_best_weights**

This pair is the single most useful default: train "too long," stop when validation stops improving, and roll weights back to the best epoch automatically.

Writing a custom callback

7. Common Architectures

7.1 MLP (tabular / flattened)

PYTHON

```
mlp = keras.Sequential([
    layers.Input(shape=(n_features,)),
    layers.Dense(128, activation="relu"),
    layers.BatchNormalization(),
    layers.Dropout(0.3),
    layers.Dense(1, activation="sigmoid"),   # binary output
])
```

7.2 CNN (images)

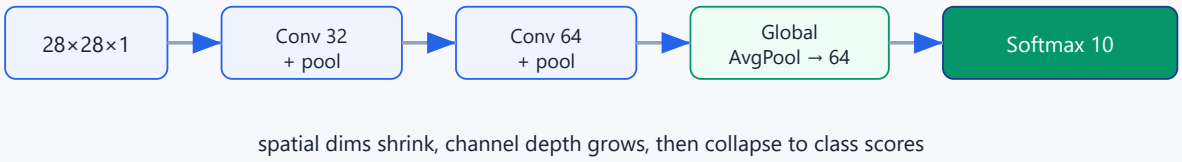
PYTHON

```
cnn = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, 3, activation="relu", padding="same"),
    layers.MaxPooling2D(),                  # 28x28 → 14x14
    layers.Conv2D(64, 3, activation="relu", padding="same"),
    layers.MaxPooling2D(),                  # 14x14 → 7x7
    layers.GlobalAveragePooling2D(),        # 7x7x64 → 64
    layers.Dense(10, activation="softmax"),
])
```

7.3 RNN (sequences / text)

PYTHON

```
rnn = keras.Sequential([
    layers.Input(shape=(maxlen,)),          # integer token ids
    layers.Embedding(input_dim=vocab, output_dim=128),
    layers.Bidirectional(layers.LSTM(64)),  # reads forward + backward
    layers.Dense(1, activation="sigmoid"),  # sentiment
])
```



spatial dims shrink, channel depth grows, then collapse to class scores

Fig 4 — A typical CNN: convolution + pooling blocks reduce spatial resolution while increasing feature depth.

8. Regularization: Fighting Overfitting

Deep nets memorize. Regularization constrains capacity so the model generalizes. Keras offers several complementary tools you can stack.

DROPOUT

`layers.Dropout(0.3)` randomly zeros activations during training, forcing redundancy.

WEIGHT DECAY (L2)

`Dense(64, kernel_regularizer=keras.regularizers.l2(1e-4))` penalizes large weights.

BATCHNORM

`layers.BatchNormalization()` stabilizes and mildly regularizes by normalizing activations.

EARLY STOPPING

Stop before the model overfits — the cheapest regularizer of all.

PYTHON

```python
from keras import regularizers

layer = layers.Dense(
    128, activation="relu",
    kernel_regularizer=regularizers.l2(1e-4),  # L2 weight penalty
)
# Data augmentation (images) is regularization too:
aug = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
])
```

⚠ **Don't stack blindly**

BatchNorm and Dropout can interact badly if ordered carelessly. A common safe recipe is Dense → BatchNorm → Activation → Dropout. Validate on held-out data rather than trusting a formula.

9. Saving, Loading, and Deployment

Keras 3's native format is a single `.keras` zip archive containing architecture, weights, and optimizer state. Prefer it over the legacy HDF5 (`.h5`) and the raw TF SavedModel directory for most workflows.

PYTHON

```python
# Save the full model (architecture + weights + optimizer state)
model.save("model.keras")
restored = keras.models.load_model("model.keras")

# Save ONLY weights (needs matching architecture to load)
model.save_weights("weights.weights.h5")
model.load_weights("weights.weights.h5")

# Export a backend-native inference artifact (e.g. TF SavedModel) for serving
model.export("saved_model_dir")   # inference-only, no training state
```

Format	Call	Use when
`.keras`	`model.save()`	Default: resume training or reload exactly
Weights `.weights.h5`	`save_weights()`	Transfer weights into a known architecture
Exported artifact	`model.export()`	Production serving / TF Serving / TFLite

10. Worked Mini-Example: MNIST End-to-End

A complete, runnable classifier that ties the lifecycle together — load data, build, compile, train with callbacks, evaluate, and save.

PYTHON

```python
import keras
from keras import layers

keras.utils.set_random_seed(42)

# 1) Data: 60k training, 10k test grayscale digits
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train = x_train.astype("float32") / 255.0   # scale to [0, 1]
x_test = x_test.astype("float32") / 255.0
x_train = x_train[..., None]                  # add channel dim → (N,28,28,1)
x_test = x_test[..., None]

# 2) Model: small CNN
model = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, activation="relu"),
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.3),
    layers.Dense(10, activation="softmax"),
])

# 3) Compile
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

# 4) Train with early stopping
model.fit(x_train, y_train, validation_split=0.1, epochs=15, batch_size=128,
          callbacks=[keras.callbacks.EarlyStopping(patience=3,
                                                   restore_best_weights=True)])

# 5) Evaluate and save
print("test accuracy:", model.evaluate(x_test, y_test)[1])
model.save("mnist_cnn.keras")
```

11. Cheat Sheet

Need	Code
Inspect model	`model.summary()`
Count params	`model.count_params()`
Freeze a layer	`layer.trainable = False`
Custom metric	`metrics=[keras.metrics.AUC()]`
Class imbalance	`fit( ... , class_weight={0:1., 1:5.})`
LR schedule	`optimizers.Adam(CosineDecay( ... ))`
Mixed precision	`keras.mixed_precision.set_global_policy("mixed_float16")`
Switch backend	`export KERAS_BACKEND=jax`
Reload model	`keras.models.load_model("m.keras")`
Plot training	`history.history["val_loss"]`

Keras 3 vs legacy tf.keras

Aspect	Keras 3 (`import keras`)	Legacy `tf.keras`
Backends	TensorFlow, JAX, PyTorch	TensorFlow only
Save format	`.keras` (default)	`.h5` / SavedModel
Low-level ops	`keras.ops.*` (backend-agnostic)	`tf.*`
Status	Actively developed	Maintenance / compatibility

12. Best Practices and Common Pitfalls

✓ **Do**

Start with the simplest API that fits (Sequential before Functional before subclassing). Scale inputs to a sane range. Always keep a validation split and use `EarlyStopping`. Save in `.keras` format.

✦ **Do**

Bake preprocessing (Normalization, Rescaling, TextVectorization) into the model so training and serving use identical transforms. Use `keras.utils.set_random_seed()` for reproducible runs.

⚠ **Watch out**

`sparse_categorical_crossentropy` expects integer labels; `categorical_crossentropy` expects one-hot. Mixing them silently trains a broken model. Match the loss to your label format.

✕ **Avoid**

Forgetting to pass the `training` flag in a subclassed `call()` — Dropout/BatchNorm then misbehave at inference. Also avoid setting `KERAS_BACKEND` *after* `import keras`; the backend locks at import time.

13. Interview Questions & Answers

What is the difference between the Sequential and Functional APIs?

Sequential is a strictly linear stack with one input and one output — you cannot express branches, skip connections, or multiple inputs. The Functional API treats layers as callables on symbolic tensors, letting you wire arbitrary directed acyclic graphs: multi-input/output models, shared layers, and residual connections. Use Sequential for simple stacks and Functional for anything with topology.

What does `compile()` actually do?

It configures the training process without running it: it attaches the optimizer, the loss function, and the metrics, and prepares the (backend-specific) train step. No weights change and no data

flows until `fit()` is called.

When would you choose `sparse_categorical_crossentropy` over `categorical_crossentropy`?

Both are for multiclass classification with a softmax output. Use the sparse version when labels are integer class indices (e.g. `3`); use the plain version when labels are one-hot vectors (e.g. `[0,0,0,1, ...]`). The sparse form avoids materializing one-hot vectors and is the common default.

How does Dropout behave differently in training vs inference?

During training it randomly zeros a fraction `rate` of activations and scales the rest up by `1/(1-rate)` (inverted dropout) so expected magnitude is preserved. At inference it is a no-op — all units pass through. Keras switches modes via the `training` flag, handled automatically in Sequential/Functional models.

What is the role of callbacks, and name three important ones.

Callbacks hook into the training loop at defined events to add behavior without changing model code. `EarlyStopping` halts when a monitored metric stops improving; `ModelCheckpoint` saves the best model to disk; `ReduceLROnPlateau` lowers the learning rate when progress stalls. Others include `TensorBoard` and `LearningRateScheduler`.

Explain the multi-backend feature of Keras 3.

Keras 3 defines layers and models against an internal backend-agnostic operations layer (`keras.ops`). The actual tensor math runs on TensorFlow, JAX, or PyTorch, selected by the `KERAS_BACKEND` env var at import time. The same model code trains and runs on any of the three, letting teams pick a backend for performance, ecosystem, or deployment reasons without rewriting models.

Why bake preprocessing layers into the model instead of transforming data beforehand?

Because the saved model then contains the exact transform, eliminating train/serve skew — the production system cannot accidentally use a different scaling or vocabulary. It also simplifies deployment since clients send raw inputs. `Normalization.adapt()` computes statistics from training data and freezes them into the layer.

How do you prevent overfitting in a Keras model?

Combine techniques: add Dropout and/or L2 weight regularization, use BatchNormalization, apply data augmentation for images, reduce model capacity, gather more data, and use EarlyStopping with a validation split. Monitor the gap between training and validation loss to confirm the fix.

What is the difference between `model.save()`, `save_weights()`, and `export()`?

> `save()` writes a `.keras` archive with architecture, weights, and optimizer state — you can resume training. `save_weights()` stores only tensors, requiring you to rebuild the architecture before loading. `export()` produces a backend-native inference-only artifact (like a TF SavedModel) suitable for serving, with no training state.

> **What does `return_sequences=True` do in an LSTM layer?**
>
> By default an LSTM returns only its final hidden state (shape `(batch, units)`). With `return_sequences=True` it returns the output at every timestep (shape `(batch, timesteps, units)`), which is required when stacking recurrent layers or feeding a per-timestep output layer (e.g. sequence labeling).

> **How are a layer's weights created — eagerly or lazily?**
>
> Lazily by default. A layer builds its weights the first time it is called on real input (or when it can infer the input shape, e.g. via a preceding `Input`). This is why `model.summary()` may report an unbuilt model until it sees an input shape. Passing `keras.Input(shape=...)` triggers eager building.

> **What is `GlobalAveragePooling2D` and why use it instead of `Flatten`?**
>
> It averages each feature map down to a single value, turning a `(H,W,C)` tensor into `(C,)`. Compared to `Flatten` + Dense, it drastically reduces parameters (no giant dense weight matrix), acts as a structural regularizer, and makes the network tolerant to input size — common in modern CNN classifier heads.

14. Further Learning

- Official Keras 3 guides and API docs at `keras.io` — especially the "Functional API" and "Making new layers & models via subclassing" guides.
- KerasCV and KerasHub (formerly KerasNLP) for prebuilt vision and transformer/LLM models with pretrained weights.
- Write a custom training loop with `train_step()` override, then compare across backends (GradientTape on TF, functional grads on JAX).
- Learn mixed-precision and multi-GPU/TPU distribution with `keras.distribution`.
- Deep-dive the companion guides in this series: TensorFlow, deep learning foundations, and MLOps for serving Keras models in production.